



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
INSTITUTO DE INGENIERÍA MATEMÁTICA Y COMPUTACIONAL
PROCESAMIENTO DE DATOS MASIVOS

Tarea 2

Julio 2026

Primer Semestre 2026

Benjamín Thareau - Ignacio Moya

Parte 1: Recuperación Densa (Dense RAG)

La primera parte implementa un RAG denso clásico. Para cada pregunta se calcula un embedding con `intfloat/multilingual-e5-base`; luego se consulta MillenniumDB por los nodos `Embedding` más cercanos usando distancia coseno:

```
dist = COSINE_DISTANCE(?emb.value, tensorFloat(vector_str))
```

La consulta MQL retorna el identificador del chunk, la intervención, el texto y metadatos del orador, partido y rol parlamentario. Como MillenniumDB puede devolver filas aplanadas, el código normaliza cada fila con un parser basado en `csv.reader`, evitando que comas dentro del texto rompan la extracción de columnas.

El flujo denso funcionó correctamente para preguntas abiertas donde la similitud semántica es suficiente. Por ejemplo, ante la pregunta sobre reforma de pensiones, los primeros resultados incluyeron las intervenciones `i54833`, `i61028`, `i64958`, `i54784` e `i54814`, con distancias coseno entre 0.1554 y 0.1592. A partir de esos chunks el LLM pudo sintetizar argumentos sobre pensiones de expresidentes, pensión de sobrevivencia, equidad y carga institucional.

La limitación principal aparece en preguntas que piden una atribución categórica. En la pregunta “¿Qué postura tiene la UDI respecto al aumento del salario mínimo?”, el recuperador denso trajo contexto de varios partidos: Partido Socialista, UDI y Partido Comunista. Esto no es un error matemático del embedding, sino una consecuencia de ordenar solo por similitud semántica: discursos de partidos distintos pueden parecer muy cercanos si tratan el mismo tema.

Parte 2: GraphRAG sobre MillenniumDB

Para resolver preguntas donde importa la estructura legislativa, se implementó un recuperador GraphRAG. La idea es combinar la cercanía semántica de los embeddings con restricciones explícitas sobre el grafo: intervención, embedding, posición parlamentaria, persona, partido, sesión y periodo legislativo.

Patrón A: restricción tipada

El primer patrón filtra antes de ordenar por distancia coseno. Se implementaron filtros por partido y por cámara/rol parlamentario. El filtro por partido atraviesa:

Intervention $\rightarrow$ Position $\rightarrow$ PoliticalParty

El filtro por cámara usa el atributo `Position.role`, mapeando valores como “Senado” o “senadores” a `Senador`, y valores de Cámara de Diputados a `Diputado`. Así, si la pregunta pide una postura de un partido o cámara específica, el LLM solo recibe chunks que cumplen esa condición estructural.

Patrón B: agregación y contraste

El segundo patrón recupera un conjunto semántico más amplio y luego agrupa por metadatos del grafo. Se usó para comparar posturas por partido y por rol parlamentario. En la pregunta sobre migración irregular y seguridad fronteriza, el sistema formó grupos para Renovación Nacional, Democracia Cristiana, UDI, Partido Socialista y Partido Republicano. En la pregunta sobre salud mental, separó resultados de `Senador` y `Diputado`.

Este patrón permite que el LLM conteste comparando posturas sin mezclar todo en un único bloque de contexto. La agrupación también hace más auditable la respuesta, porque se puede revisar qué intervenciones alimentaron cada grupo.

Patrón C: atributos temporales y numéricos

El tercer patrón agrega condiciones sobre propiedades internas del grafo. Se implementaron dos casos:

- **Cohortes por fecha de nacimiento:** filtro sobre `Person.birth_date`. Para cambio climático y medio ambiente se compararon parlamentarios nacidos entre 1975-1995 e integrantes nacidos entre 1940-1960. En la primera cohorte aparecieron, por ejemplo, intervenciones de `i55123` e `i54713`; en la segunda, `i63899`, `i60046` e `i58804`.
- **Rangos de sesión:** filtro sobre `Session.date`. Para violencia en La Araucanía se compararon sesiones de 2025 y 2026. El sistema recuperó, por ejemplo, `i54966` y `i52862` para 2025, e `i56338` y `i52862` para 2026.

Comparación GraphRAG vs Dense RAG

La comparación más clara fue la pregunta “¿Qué opina la UDI sobre el impuesto a las grandes fortunas?”. El recuperador denso trajo un contexto mixto con Partido Humanista, Renovación Nacional, Partido Socialista y UDI. En cambio, GraphRAG aplicó primero el filtro `Position -> PoliticalParty` y entregó solo intervenciones de la UDI, incluyendo i58600, i62299, i60901 e i64798.

Esto muestra una ventaja concreta de GraphRAG: cuando la pregunta contiene una restricción estructural, el grafo reduce la contaminación de contexto antes de que el LLM redacte la respuesta. El método denso puede encontrar discursos temáticamente cercanos, pero no garantiza pertenencia al partido solicitado.

Parte 3: Recuperación Híbrida (Sparse + Dense)

La tercera parte agrega un recuperador *sparse* propio para mejorar preguntas con términos exactos, acrónimos, nombres de ley o boletines. Se implementó BM25 desde Python y se guardó el índice en `part3/sparse_store/bm25_index.pkl.gz`. Los documentos del índice mantienen los mismos identificadores de intervención y chunk usados por el recuperador denso, por lo que ambos rankings se pueden fusionar sin perder trazabilidad.

Índice BM25 y preprocesamiento

El constructor del índice extrae intervenciones desde MillenniumDB, descarta registros sin texto y conserva identificadores existentes; solo usa un identificador de contingencia si falta el ID original. Para evitar ejecuciones excesivamente pesadas, el índice usa un límite configurable y por defecto trabaja con hasta 3000 documentos.

El preprocesamiento transforma a minúsculas, normaliza con NFD para remover tildes, extrae tokens alfanuméricos con expresiones regulares y filtra *stopwords* frecuentes del español. El puntaje se calcula con BM25:

$$\text{Score}_{BM25} = \text{IDF} \times \frac{\text{TF} \cdot (k_1 + 1)}{\text{TF} + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

Los hiperparámetros usados fueron $k_1 = 1.5$ y $b = 0.75$.

Fusión y reranking determinístico

La estrategia híbrida obtiene candidatos del recuperador denso y del sparse, y luego aplica Reciprocal Rank Fusion (RRF):

$$\text{RRF_score} = \frac{w_{\text{dense}}}{60 + \text{rank}_{\text{dense}}} + \frac{w_{\text{sparse}}}{60 + \text{rank}_{\text{sparse}}}$$

Después se aplica un reranking determinístico inspirado en Maximal Marginal Relevance. El puntaje base combina RRF normalizado con cobertura léxica exacta:

$$\text{base} = 0.78 \cdot \text{RRF}_{\text{norm}} + 0.22 \cdot \text{lexical_overlap}$$

La selección final usa $\lambda = 0.82$ y penaliza documentos redundantes con similitud de Jaccard frente a los ya seleccionados:

$$\text{Score} = \lambda \cdot \text{base} - (1 - \lambda) \cdot \max_{s \in S} \text{Jaccard}(d, s)$$

No se usa un LLM para fusionar ni reordenar; toda la etapa de ranking es reproducible.

Resultados observados

El recuperador sparse fue especialmente útil en consultas con términos exactos. Para “CAE, condonación y FES”, BM25 ubicó como primer resultado `i54520` con los términos `cae`, `condonacion`, `educacion`, `fes` y `superior`. En la versión híbrida, `i54520` también quedó primero, combinando `sparse_rank = 1` con `dense_rank = 6`.

Para “Ley Nain-Retamal y legítima defensa de Carabineros”, el híbrido fue aún más estable: `i55753` e `i57949` quedaron en las dos primeras posiciones tanto por ranking denso como por BM25. Esto indica que, cuando ambos recuperadores coinciden, la fusión refuerza resultados robustos.

En la consulta sobre muerte digna, eutanasia y cuidados paliativos, el sparse priorizó intervenciones con coincidencias literales como `i55383`, `i55205`, `i58746`, `i62445` e `i55674`. El híbrido mezcló candidatos densos y sparse; por eso algunos resultados muestran `sparse_rank = None` o `dense_rank = None`. Ese valor no significa ausencia de información, sino que el documento entró al ranking final desde el otro recuperador y no apareció dentro del pool considerado por esa componente.

Análisis general de Recuperación

Dense RAG es una buena base para preguntas abiertas, pero no controla por sí solo restricciones de partido, cámara, fecha o cohorte. GraphRAG corrige ese problema usando la estructura del grafo antes de entregar contexto al LLM. La parte híbrida complementa ambos enfoques al recuperar mejor nombres propios, siglas y términos exactos que pueden diluirse en el espacio semántico.

En conjunto, las tres partes cumplen roles distintos: el denso maximiza cobertura semántica, GraphRAG maximiza precisión estructural y el híbrido mejora la recuperación léxica sin renunciar a similitud semántica.

Parte 4: Evaluación y Comparación

Para evaluar empíricamente el rendimiento de las distintas arquitecturas de Generación Aumentada por Recuperación (RAG), se diseñó un experimento comparativo utilizando 9 consultas complejas y evaluando manualmente la pertinencia de los fragmentos recuperados en una escala de 1 a 7.

4.1 Preguntas Diseñadas

A las 7 preguntas provistas por el equipo docente, se añadieron dos consultas diseñadas metodológicamente para exponer las debilidades particulares de los modelos semánticos y léxicos:

8. *¿Cuál es la postura específica del Partido Comunista de Chile frente al despliegue de las Fuerzas Armadas en infraestructura crítica y las Reglas de Uso de la Fuerza (RUF) en zonas urbanas?*

Objetivo: Discriminar a favor de GraphRAG. Al buscar semánticamente "Fuerzas Armadas" y "seguridad", el modelo denso se inunda de discursos de la oposición, diluyendo la postura específica del Partido Comunista.

9. *Identifica las intervenciones que debaten explícitamente la 'Ley Karin' y la ratificación del 'Convenio 190' de la OIT. ¿Qué exigencias preventivas específicas se mencionan para los empleadores?*

Objetivo: Discriminar a favor del modelo Sparse (BM25) exigiendo una coincidencia exacta de términos ("Karin", "190"), donde el modelo denso suele perder precisión al buscar abstracciones sobre acoso laboral.

4.2 y 4.3 Evaluación Manual

Se ejecutó la recuperación ($top-k = 5$) para las 9 preguntas a través de cuatro configuraciones manteniendo fija la generación mediante `gpt-4o-mini`. Se calificó manualmente cada uno de los 180 fragmentos recuperados (*chunks*) sin utilizar métodos de *LLM-as-a-judge*.

La métrica agregada utilizada para la comparación fue la **Relevancia Promedio** (Suma de notas / Cantidad de *chunks* evaluados por estrategia).

4.4 Visualización

Como se observa en la Figura 1, el motor Híbrido obtuvo el mejor desempeño general (3.25), seguido por la recuperación Densa (3.20) y GraphRAG (3.08). El motor Sparse (BM25) obtuvo el rendimiento promedio más bajo (2.60). El puntaje globalizado en el rango medio-bajo se explica por la ausencia total de documentos relevantes para ciertos cortes temporales en la base de datos (e.g., *Ley Karin* o proyectos de *Muerte Digna* archivados), lo que castigó severamente a todas las estrategias con calificaciones mínimas.

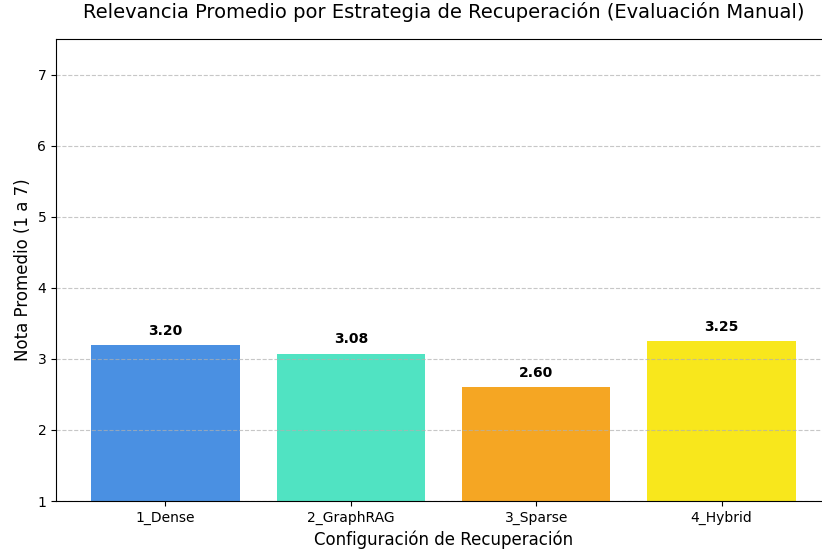


Figure 1: Relevancia Promedio por Estrategia de Recuperación (Escala 1 a 7).

4.5 Análisis

A partir de la evaluación empírica, se extraen las siguientes conclusiones sobre el comportamiento de las estrategias y las decisiones de diseño adoptadas:

- **Dominio por Estrategia:** La recuperación **Densa** demostró supremacía en consultas de alta carga conceptual y lenguaje natural abstracto (e.g., discusión sobre "CAE" o "Ley Naín-Retamal"). Por el contrario, el modelo **Sparse** fracasó en vincular conceptos semánticos, pero fue absolutamente superior en consultas de tipo "aguja en el pajar" (e.g., la consulta sobre aborto y el artículo 342 del Código Penal), donde el modelo semántico recuperaba documentos ruidosos. Respecto a **GraphRAG**, su rendimiento promedio (3.08) fue ligeramente inferior al denso puro al evaluar un flujo automatizado sin la inyección manual de metadatos estructurales (comportándose funcionalmente como un *fallback* semántico), pero demostró ser la única herramienta capaz de resolver preguntas como la Q8 al aislar limpiamente la topología de la base de datos por partido político.
- **Aporte de la Fusión y el Rerank:** El motor **Híbrido** (3.25) mitigó las deficiencias individuales. La decisión de diseño de utilizar *Reciprocal Rank Fusion* ($k = 60$) permitió combinar los vectores ortogonales de similitud coseno y TF-IDF de manera no paramétrica. A su vez, el *reranking* final mediante *Maximal Marginal Relevance* (MMR, $\lambda = 0.82$) con distancia de Jaccard fue fundamental para garantizar diversidad léxica, evitando inyectar cinco discursos idénticos al modelo de lenguaje.
- **Calidad de Recuperación frente a Generación (LLM):** Se comprobó una correlación estricta entre la pertinencia del contexto recuperado y la calidad de la respuesta

final. En las consultas Q7 y Q9, donde el límite temporal de la base de datos MillenniumDB impedía poseer los documentos solicitados, todos los recuperadores inyectaron ruido (notas 1 y 2). Ante este escenario, el LLM fue incapaz de generar una respuesta analítica válida, demostrando que la fase generativa está completamente supeditada a la veracidad de la recuperación.

- **Decisiones de Diseño:** Se optó por mantener una granularidad de *chunk* equivalente a una "intervención completa", garantizando que no se perdiera el hilo argumentativo del parlamentario. La estrategia *sparse* se construyó desde cero en Python sin librerías externas de indexación, aplicando una limpieza rigurosa de *stopwords* en español para concentrar la distribución probabilística exclusivamente en los sustantivos y verbos clave del dominio legislativo. El *prompt* del sistema instruyó explícitamente al modelo a basarse solo en el contexto, forzando la evaluación directa de la eficacia del RAG.